Preinforme - Desafío II

UdeABook - Informática II | Análisis por funcionalidad

Contexto del problema

UdeABook será una red social por consola desarrollada en C++. Permitirá iniciar sesión, administrar amistades, visitar álbumes y consultar mensajes. El trabajo se centra en organizar la información y realizar estas operaciones con memoria dinámica y clases propias. Mostrar una foto significa imprimir su ruta; no es necesario abrir la imagen. Los usuarios, álbumes y fotos ya estarán registrados en los archivos.

I. Carga y actualización de datos

Esta funcionalidad debe recuperar los datos al iniciar y conservar los cambios al terminar, sin aparecer como una opción del menú. Proponemos cargar la información en RAM y trabajar sobre los objetos creados. Así, una búsqueda o una consulta no necesita recorrer nuevamente los archivos del disco.

En usuarios.txt se usarán tres líneas por usuario: la primera con sus datos personales y credenciales, la segunda con los nombres de sus amigos y la tercera con los nombres de quienes le enviaron solicitudes. Una lista vacía conservará su línea. Se definirán separadores y una regla para representar esos caracteres cuando aparezcan dentro de un campo. Álbumes, fotos y mensajes se guardarán en archivos separados, vinculados mediante identificadores o nombres únicos.

La carga se hará en dos pasos: primero se crearán todos los usuarios y después se resolverán sus relaciones. Esto evita buscar un objeto que todavía no existe. Las amistades y solicitudes serán arreglos dinámicos de punteros; cada usuario existirá una sola vez. También se cargarán los álbumes, los datos de las fotos y los mensajes, sin cargar el contenido de las imágenes. Se revisarán nombres repetidos y referencias a usuarios inexistentes.

Al guardar se escribirán los datos actuales, incluyendo nuevas solicitudes, amistades, mensajes, estados de lectura y likes. Los punteros solo sirven dentro de la ejecución: en los TXT se guardarán identificadores. Cada arreglo tendrá cantidad y capacidad; si necesita crecer, se ampliará con espacio adicional para evitar redimensionarlo en cada inserción.

II. Ingreso y visualización de mejores amigos

Se pedirá nombre de usuario y contraseña. Para localizar la cuenta proponemos ordenar una vez un arreglo de punteros por nombre con MergeSort y aplicar búsqueda binaria. Los objetos originales conservarán su posición, por lo que sus relaciones no se dañarán. Si las credenciales son correctas se guardará un puntero al usuario activo y se mostrará el menú; de lo contrario se informará el error.

Al ingresar se seleccionarán tres amigos al azar y se imprimirán sus rutas de perfil. La elección será ponderada por amigos en común: peso 1 para hasta 10, 2,5 para entre 11 y 20, y 3,5 para más de 21. Se sumarán los pesos y se realizará un sorteo acumulado, retirando temporalmente cada elegido para no repetirlo en el grupo. Falta confirmar con el profesor el peso para exactamente 21 coincidencias.

La opción de actualizar realizará otro sorteo y evitará repetir el mismo grupo cuando exista una alternativa. Si el usuario tiene menos de tres amigos se mostrarán los disponibles; si no tiene, se indicará. No se impondrá un orden fijo y se distinguirá entre cambiar el grupo y exigir tres personas totalmente nuevas, algo imposible cuando hay pocos amigos.

Análisis por funcionalidad

UdeABook - Informática II

III. Administrar amistades

Para enviar una solicitud se recibirá el nombre del destinatario y se buscará su cuenta en el índice de usuarios. Antes de agregarla se comprobará que exista, que no sea el usuario activo, que no sean amigos y que no haya una solicitud pendiente en ninguno de los dos sentidos. Así se evitan duplicados y solicitudes cruzadas. Si todo es válido, se añadirá un puntero al solicitante en la lista de solicitudes recibidas del destinatario.

Para aceptar se mostrará la lista de solicitudes recibidas y se seleccionará una. La amistad debe agregarse a los dos usuarios, porque la relación es mutua. Primero se comprobará que ambos arreglos tengan espacio y luego se actualizarán las dos listas. Finalmente se retirará la solicitud pendiente, ajustando la cantidad de elementos sin eliminar el objeto del usuario.

Si la lista está vacía se informará y se regresará al menú. Se revisará también que la opción seleccionada sea válida. La descripción general menciona rechazar solicitudes, aunque el apartado de funcionalidades solo detalla enviar y aceptar; se consultará ese alcance. Si se incluye el rechazo, bastará con quitar la solicitud recibida sin crear ninguna amistad.

IV. Visitar el perfil de un amigo

Se pedirá el nombre de un amigo y se verificará la relación antes de mostrar su perfil. Después se recorrerán sus álbumes y se listarán únicamente los públicos o con privacidad para amigos. Los privados no se mostrarán. Al seleccionar un álbum se volverá a comprobar que corresponda a una opción visible, para evitar acceder a otro por un índice incorrecto.

El carrusel mantendrá la posición de la foto actual dentro del arreglo. Siguiente avanzará una posición y volverá a la primera después de la última; previa hará el recorrido contrario. Con una sola foto ambas opciones conservarán esa foto. Si el álbum está vacío se avisará antes de intentar acceder al arreglo. Salir terminará el recorrido y devolverá el control al menú correspondiente.

Por cada foto se imprimirá su ruta y se ofrecerán las cuatro acciones solicitadas: salir, siguiente, previa y dar o quitar like. Para alternar correctamente el like proponemos guardar referencias a los usuarios que lo dieron. Si el usuario activo ya aparece, se retira su referencia; si no aparece, se agrega. El conteo se obtendrá de esa cantidad, evitando likes repetidos o valores negativos.

V. Ver sugerencias de amistad

Se solicitará un entero K entre 1 y 20. Los candidatos se obtendrán recorriendo los amigos de los amigos del usuario activo. Se excluirán el propio usuario y sus amigos actuales. Cada candidato aparecerá una sola vez y tendrá un contador de amigos en común. Si se encuentra por medio de otro amigo, aumentará su contador en lugar de agregarse de nuevo.

Los candidatos se ordenarán de mayor a menor coincidencia con un algoritmo propio; en los empates se podrá usar el nombre para obtener un resultado estable. Se mostrarán los primeros K o todos los disponibles si hay menos. Desde esa lista se podrá enviar una solicitud aplicando las mismas validaciones de la funcionalidad III. Una solicitud pendiente se indicará para evitar intentar enviarla otra vez.

Aunque la guía sugiere investigar la matriz de adyacencia, proponemos usar los arreglos de amigos ya existentes. Así no se reserva una casilla para cada pareja posible de usuarios. Para acumular coincidencias puede usarse un arreglo temporal de contadores asociado al índice de usuarios, que se liberará al finalizar la consulta.

Análisis por funcionalidad

UdeABook - Informática II

VI. Administrar mensajes

Cada mensaje será un único objeto con contenido, fecha de envío, estado de lectura y referencias a quien lo envía y a quien lo recibe. No se crearán copias distintas para mensajes enviados y recibidos. RedSocial administrará estos objetos y las consultas seleccionarán los que correspondan al usuario activo.

Enviar: se pedirá el nombre de un amigo y el texto. Se comprobará que el destinatario exista y pertenezca a las amistades del usuario activo. El contenido no podrá estar vacío ni superar 512 caracteres. Si cumple las condiciones se creará el mensaje, se asignará la fecha y quedará marcado como no leído. Si alguna validación falla, no se agregará ningún objeto.

Leer nuevos: se seleccionarán los mensajes cuyo destinatario sea el usuario activo y cuyo estado sea no leído. Se ordenarán por fecha de envío y se mostrarán con el nombre de quien los envió. Después de mostrar cada uno se cambiará su estado a leído. Si no hay mensajes nuevos se informará, sin alterar otros mensajes.

Buscar por fechas: se solicitarán una fecha inicial y una final. Se validarán los días, meses y años, y que el inicio no sea posterior al final. Se listarán los mensajes recibidos cuya fecha esté dentro del intervalo, incluyendo ambos extremos. La comparación se hará por año, mes y día, no por el orden del texto de la fecha. Esta consulta no cambiará por sí sola el estado de lectura.

Una primera solución recorrerá el arreglo general de mensajes para seleccionar los recibidos por el usuario. Su costo crece con el total de mensajes, aunque ese usuario tenga pocos. Si las mediciones muestran que este recorrido es costoso, se podrá añadir un índice de punteros por destinatario, construido durante la carga y actualizado al enviar. El índice ocuparía memoria adicional, pero conservaría un solo objeto por mensaje.

VII. Medición del consumo de recursos

La clase Recursos llevará el registro de iteraciones y memoria. Al comenzar cada funcionalidad se reiniciará su contador de iteraciones y se contarán tanto los ciclos directos como los ejecutados por los métodos llamados. Por ejemplo, buscar un usuario, revisar sus amistades y ordenar resultados forman parte del costo de la operación que los necesita. Si se utiliza un componente externo, como el generador aleatorio permitido, se reportará su uso.

La memoria se calculará con los objetos y estructuras que existan en el instante medido. Se incluirán los arreglos según su capacidad reservada, los índices auxiliares y las variables locales y parámetros por valor que sigan activos. Un puntero no equivale a una copia del usuario: se contará el espacio del puntero y el objeto se contará una sola vez. Tampoco se sumará el tamaño de los archivos de imagen, porque solo se guardan sus rutas.

El registro aumentará al reservar memoria y disminuirá al liberarla. sizeof permitirá conocer la parte fija de un objeto, pero será necesario sumar sus reservas dinámicas sin contarlas dos veces. Las métricas se mostrarán al terminar cada operación, tomando la medida antes de que desaparezcan las variables locales que deban incluirse. En este preinforme se propone el procedimiento; los resultados se obtendrán durante la implementación.

Diagrama de clases

Atributos y métodos principales

